

Neural Networks

HW/SW Codesign - Week 6

Note: This PDF contains corrections made after the lecture. In particular, the section about hyperparameter tuning was moved to week 7.

Today's Topics

- Theory review
 - Models
 - Cost functions
 - Gradient descent
- The fully connected neural network
 - The neuron
 - Layers
 - Activation functions
- Training neural networks
 - Backpropagation
 - Initialization
 - Vanishing and exploding gradients
 - Batch normalization
 - Momentum and the Adam optimizer
 - Regularization
- Transfer learning

The Fully Connected Neural Network

The vanilla neural network

The Neuron

- Logistic regression:

$$\hat{y} = \sigma(\theta^T \mathbf{x}), \quad \mathbf{x} = [1, x_1, \dots, x_n]^T$$

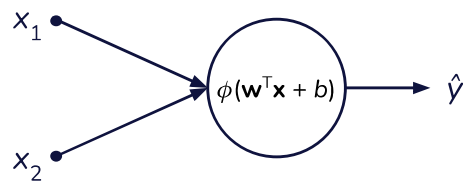
- Split θ into **weights** $\mathbf{w}$ and **bias** b :

$$\hat{y} = \sigma(\mathbf{w}^T \mathbf{x} + b), \quad \mathbf{x} = [x_1, \dots, x_n]^T$$

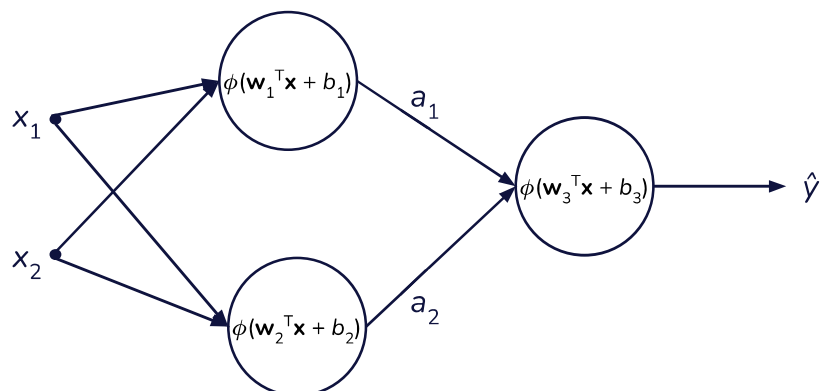
- Allow any **activation function** ϕ instead of σ :

$$\hat{y} = \phi(\mathbf{w}^T \mathbf{x} + b), \quad \mathbf{x} = [x_1, \dots, x_n]^T$$

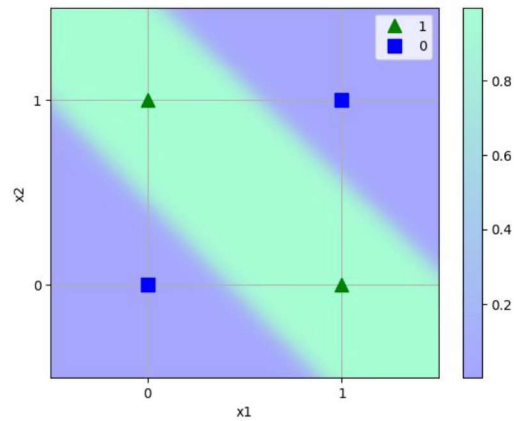
The Neuron



Neural Network



The XOR Problem with Neural Network



The XOR Problem

- Solution to the XOR problem:

$$\mathbf{a} = \sigma\left(\begin{bmatrix} 6.0 & 6.0 \\ 6.0 & 6.0 \end{bmatrix} \mathbf{x} + \begin{bmatrix} -3.0 \\ -9.0 \end{bmatrix}\right)$$
$$\hat{y} = \sigma([6.0 \quad -6.0] \mathbf{a} - 3.0)$$

Fully Connected Layer

The **fully connected layer**, also known as **dense layer**:

$$\mathbf{a} = \phi(\mathbf{W}\mathbf{x} + \mathbf{b})$$

$\mathbf{x} \in R^n$ is the feature vector,

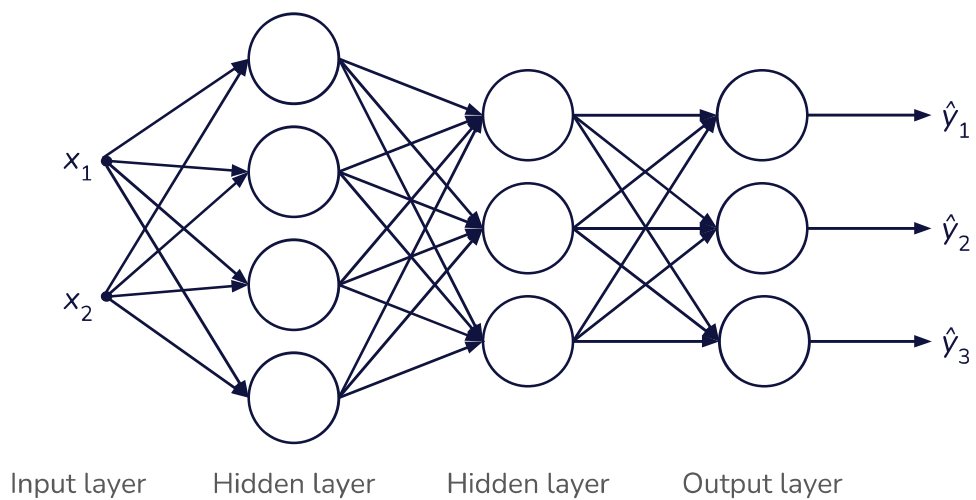
$\mathbf{W} \in R^{k \times n}$ are the weights,

$\mathbf{b} \in R^k$ are the biases,

$\phi(\cdot)$ is the activation function,

$\mathbf{a} \in R^k$ is the activation.

Fully Connected Neural Network



Fully Connected Neural Network

The **fully connected neural network**, aka **multi-layer perceptron** (MLP):

$$\begin{aligned}\mathbf{a}^{(1)} &= \phi_1(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}) \\ \mathbf{a}^{(2)} &= \phi_2(\mathbf{W}^{(2)}\mathbf{a}^{(1)} + \mathbf{b}^{(2)}) \\ &\dots \\ \hat{\mathbf{y}} &= \phi_L(\mathbf{W}^{(L)}\mathbf{a}^{(L-1)} + \mathbf{b}^{(L)})\end{aligned}$$

$\mathbf{x} \in R^n$ is the feature vector,
 $\mathbf{W}^{(l)}$ are the weights for layer l ,
 $\mathbf{b}^{(l)}$ are the biases for layer l ,
 $\phi_l(\cdot)$ is the activation function for layer l ,
 $\mathbf{a}^{(l)}$ is the activation for layer l ,
 $\hat{\mathbf{y}} \in R^k$ is the output.

Fully Connected Neural Network

- Layer weights $\mathbf{W}^{(l)}$ are 2D matrices
 - Number of neurons $\times$ number of layer inputs
- Layer biases $\mathbf{b}^{(l)}$ are vectors (1D)
 - Number of neurons
- Inputs $\mathbf{x}$ and activations $\mathbf{a}^{(l)}$ are vectors (1D)
 - $\Rightarrow$ Images and other 2D feature matrices must be **flattened** to 1D
 - Example: 28x28 image $\Rightarrow$ 784 features

The Universal Approximation Theorem

A **neural network** with **at least one hidden layer** and a **non-polynomial activation function** can approximate any continuous function $f: \mathbb{R}^n \mapsto \mathbb{R}^k$ to any desired degree of accuracy.

Activation Functions

- Linear activation
- Sigmoid
- Softmax
- ReLU
- SiLU/Swish

Linear Activation

- The identity function:

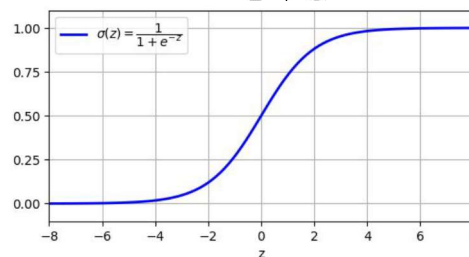
$$I(z) = z$$

- Used only in the output layer for regression problems
- Loss function: Squared error

Sigmoid Activation

- The sigmoid function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



- Often used in the output layer for binary classification problems
- Loss function: Log loss

Softmax Activation

- The softmax function:

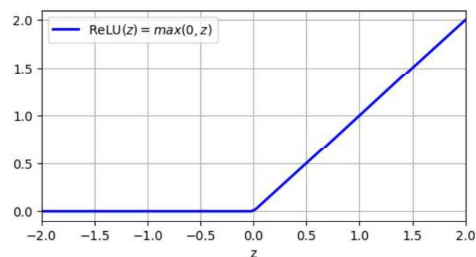
$$\text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

- Often used in the output layer for multi-class classification problems
- Loss function: Cross-entropy loss

ReLU Activation

- The ReLU (Rectified Linear Unit) function:

$$\text{ReLU}(z) = \max(0, z)$$

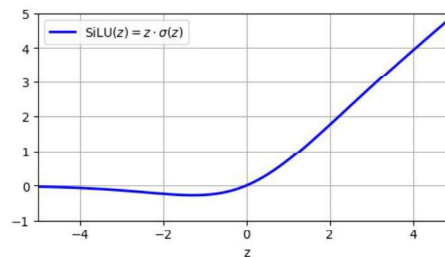


- Often used in hidden layers

SiLU/Swish Activation

- The SiLU (Sigmoid Linear Unit) function:

$$\text{SiLU}(z) = \frac{z}{1 + e^{-z}}$$



- Used in hidden layers
- Not commonly used for resource-constrained devices

Training Neural Networks

Gradient descent through the layers

Example Neural Network

- Fully connected, one hidden layer, binary classification:

$$\mathbf{a}^{(1)} = \text{ReLU}\left(\begin{bmatrix} 1.0 & 1.0 \\ 1.0 & 1.0 \end{bmatrix} \mathbf{x} + \begin{bmatrix} 0.0 \\ -2.0 \end{bmatrix}\right)$$
$$\hat{y} = \sigma\left(\begin{bmatrix} 1.0 & 2.0 \end{bmatrix} \mathbf{a}^{(1)} - 1.0\right)$$

Cost Function

- Average log loss** (fully connected neural network, binary classification):

$$J(\mathbf{W}^{(1)}, \mathbf{b}^{(1)}, \mathbf{W}^{(2)}, \mathbf{b}^{(2)}) = -\frac{1}{m} \sum_{i=1}^m (y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i))$$

where

$$\mathbf{a}^{(1)} = \text{ReLU}(\mathbf{W}^{(1)} \mathbf{x} + \mathbf{b}^{(1)})$$

$$\hat{y} = \sigma(\mathbf{W}^{(2)} \mathbf{a}^{(1)} + \mathbf{b}^{(2)})$$

- We need to compute the gradient:

$$\nabla J = \left(\frac{\partial J}{\partial \mathbf{W}^{(1)}}, \frac{\partial J}{\partial \mathbf{b}^{(1)}}, \frac{\partial J}{\partial \mathbf{W}^{(2)}}, \frac{\partial J}{\partial \mathbf{b}^{(2)}} \right)$$

Loss Function

- Log loss:

$$L(\mathbf{W}^{(1)}, \mathbf{b}^{(1)}, \mathbf{W}^{(2)}, \mathbf{b}^{(2)}) = -(y \log(\hat{y}) + (1-y) \log(1-\hat{y}))$$

where

$$\mathbf{a}^{(1)} = \text{ReLU}(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$$

$$\hat{y} = \sigma(\mathbf{W}^{(2)}\mathbf{a}^{(1)} + \mathbf{b}^{(2)})$$

- We need to compute the gradient:

$$\nabla L = \left(\frac{\partial L}{\partial \mathbf{W}^{(1)}}, \frac{\partial L}{\partial \mathbf{b}^{(1)}}, \frac{\partial L}{\partial \mathbf{W}^{(2)}}, \frac{\partial L}{\partial \mathbf{b}^{(2)}} \right)$$

Loss Function

- Log loss:

$$\mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}$$

$$\mathbf{a}^{(1)} = \text{ReLU}(\mathbf{z}^{(1)})$$

$$\mathbf{z}^{(2)} = \mathbf{W}^{(2)}\mathbf{a}^{(1)} + \mathbf{b}^{(2)}$$

$$\hat{y} = \sigma(\mathbf{z}^{(2)})$$

$$L = -(y \log(\hat{y}) + (1-y) \log(1-\hat{y}))$$

- We need to compute the gradient:

$$\nabla L = \left(\frac{\partial L}{\partial \mathbf{W}^{(1)}}, \frac{\partial L}{\partial \mathbf{b}^{(1)}}, \frac{\partial L}{\partial \mathbf{W}^{(2)}}, \frac{\partial L}{\partial \mathbf{b}^{(2)}} \right)$$

Forward Pass

- Compute everything for the training example ($\mathbf{x} = [1.0, 0.0]^T$, $y = 1.0$):

$$\begin{aligned}\mathbf{z}^{(1)} &= \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)} &&= \begin{bmatrix} 1.0 \\ -1.0 \end{bmatrix} \\ \mathbf{a}^{(1)} &= \text{ReLU}(\mathbf{z}^{(1)}) &&= \begin{bmatrix} 1.0 \\ 0.0 \end{bmatrix} \\ z^{(2)} &= \mathbf{W}^{(2)}\mathbf{a}^{(1)} + b^{(2)} &&= 0.0 \\ \hat{y} &= \sigma(z^{(2)}) &&= 0.5 \\ L &= -(y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})) &&= 0.69\end{aligned}$$

The Chain Rule

The **chain rule**: The derivative of $z(y(x))$ with respect to x is:

$$\frac{dz}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx}$$

- The chain rule is used in the backward pass to compute the partial derivatives of the loss function L with respect to the parameters $\mathbf{W}^{(1)}$, $b^{(1)}$, $\mathbf{W}^{(2)}$, $b^{(2)}$, etc.
- With the chain rule, the full partial derivatives are broken into products of derivatives of the steps of the loss computation.
- It is advantageous to start from the end and work backwards, since results can be reused.

Backward Pass

$$\frac{\partial L}{\partial b^{(2)}} = \frac{\partial L}{\partial z^{(2)}} \cdot \frac{\partial z^{(2)}}{\partial b^{(2)}} = (\hat{y} - y) \cdot 1 = -0.5$$

$$z^{(1)} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)} = \begin{bmatrix} 1.0 \\ -1.0 \end{bmatrix}$$

$$\mathbf{a}^{(1)} = \text{ReLU}(\mathbf{z}^{(1)}) = \begin{bmatrix} 1.0 \\ 0.0 \end{bmatrix}$$

$$\frac{\partial L}{\partial \mathbf{W}^{(2)}} = \frac{\partial L}{\partial z^{(2)}} \cdot \frac{\partial z^{(2)}}{\partial \mathbf{W}^{(2)}} = (\hat{y} - y) \cdot \mathbf{a}^{(1)} = \begin{bmatrix} -0.5 \\ 0.0 \end{bmatrix}$$

$$z^{(2)} = \mathbf{W}^{(2)}\mathbf{a}^{(1)} + b^{(2)} = 0.0$$

$$\hat{y} = \sigma(z^{(2)}) = 0.5$$

$$L = -(y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})) = 0.69$$

$$\frac{\partial L}{\partial \mathbf{b}^{(1)}} = \frac{\partial L}{\partial z^{(2)}} \cdot \frac{\partial z^{(2)}}{\partial \mathbf{a}^{(1)}} \cdot \frac{\partial \mathbf{a}^{(1)}}{\partial \mathbf{z}^{(1)}} \cdot \frac{\partial \mathbf{z}^{(1)}}{\partial \mathbf{b}^{(1)}} = (\hat{y} - y) \cdot \mathbf{W}^{(2)} \cdot \mathbf{1}_{z^{(1)} > 0} \cdot 1 = \begin{bmatrix} -0.5 & -1.0 \end{bmatrix}$$

$$\frac{\partial L}{\partial \mathbf{W}^{(1)}} = \frac{\partial L}{\partial z^{(2)}} \cdot \frac{\partial z^{(2)}}{\partial \mathbf{a}^{(1)}} \cdot \frac{\partial \mathbf{a}^{(1)}}{\partial \mathbf{z}^{(1)}} \cdot \frac{\partial \mathbf{z}^{(1)}}{\partial \mathbf{W}^{(1)}} = (\hat{y} - y) \cdot \mathbf{W}^{(2)} \cdot \mathbf{1}_{z^{(1)} > 0} \cdot \mathbf{x} = \begin{bmatrix} -0.5 & 0.0 \\ 0.0 & 0.0 \end{bmatrix}$$

Backpropagation

1. For all training examples in the mini-batch:
 - a. **Forward pass.** Compute the pre-activations, activations, and loss.
 - Remember all values for use in the backward pass.
 - b. **Backward pass.** Compute the partial derivatives of the loss wrt to all parameters.
 - Start with the last layer and propagate backwards, reusing $\partial L / \partial \mathbf{z}^{(l)}$.
2. Compute the averages of the partial derivatives over all examples to obtain the cost gradient.
3. Update the weights with gradient descent.

Difficulties with Training Deep Neural Networks

- Weight symmetry
- Vanishing/exploding gradients
- Ill-conditioned cost function
- Overfitting

Weight Symmetry

- Problem:
 - **Weight symmetry:** Weights in a layer stay identical if initialized with zeros
- Solution:
 - **Initialize weights with small random numbers**
 - Handled automatically with Keras and PyTorch
 - Consequence: Repeated training may give different result

Vanishing and Exploding Gradients

- Partial derivatives are products of partial derivatives for each step:

$$\frac{\partial L}{\partial \mathbf{W}^{(1)}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial \mathbf{z}^{(L)}} \cdot \frac{\partial \mathbf{z}^{(L)}}{\partial \mathbf{a}^{(L-1)}} \cdots \frac{\partial \mathbf{z}^{(2)}}{\partial \mathbf{a}^{(1)}} \cdot \frac{\partial \mathbf{a}^{(1)}}{\partial \mathbf{z}^{(1)}} \cdot \frac{\partial \mathbf{z}^{(1)}}{\partial \mathbf{W}^{(1)}}$$

- Factors generally less than 1 $\Rightarrow$ **Vanishing gradients**: $\partial L / \partial \mathbf{W}$ tends to 0
- Factors generally greater than 1 $\Rightarrow$ **Exploding gradients**: $\partial L / \partial \mathbf{W}$ tends to ∞

Vanishing and Exploding Gradients

- Problem:
 - Vanishing gradients $\Rightarrow$ Early layer parameters don't update
 - Exploding gradients $\Rightarrow$ Early layer parameters diverge
- Solutions:
 - Use ReLU-like activation function for hidden layers
 - Proper random initialization of weights
 - Skip connections (next week)
 - Batch normalization

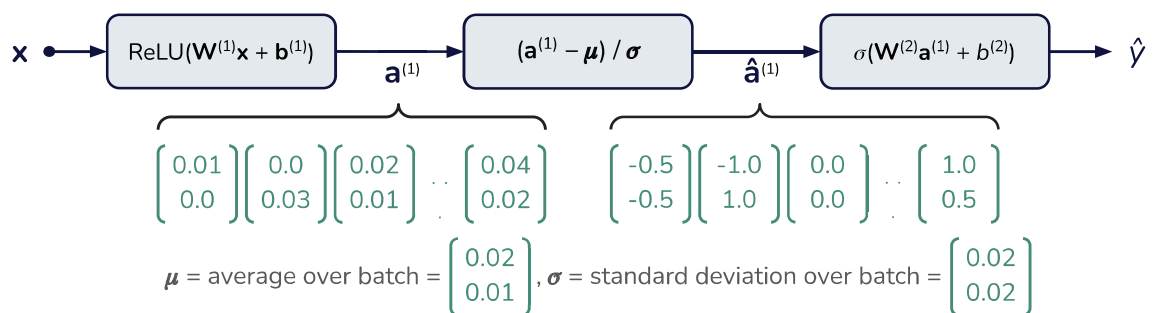
Batch Normalization

- **Batch normalization** layer:

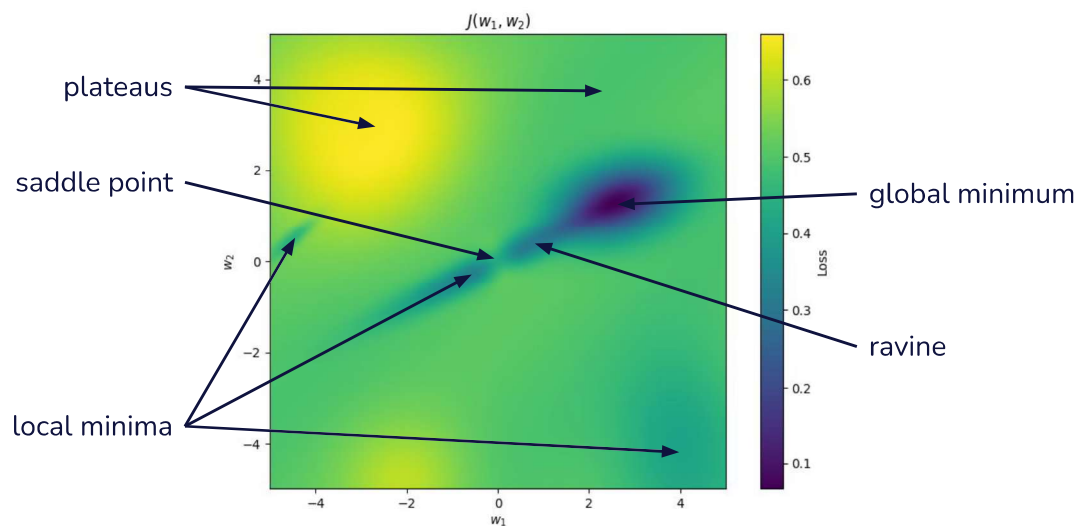
$$\hat{\mathbf{a}} = \gamma \odot \frac{\mathbf{a} - \boldsymbol{\mu}}{\sqrt{\boldsymbol{\sigma}^2 + \epsilon}} + \boldsymbol{\beta}$$

$\mathbf{a} \in R^n$ is the activation from the previous layer,
 $\boldsymbol{\mu} \in R^n$ is the average of $\mathbf{x}$ over the batch,
 $\boldsymbol{\sigma} \in R^n$ is the standard deviation of $\mathbf{x}$ over the batch,
 ϵ is a small number like 10^{-3} or 10^{-5} ,
 $\boldsymbol{\gamma} \in R^n$ are scaling parameters,
 $\boldsymbol{\beta} \in R^n$ are shifting parameters,
 $\hat{\mathbf{a}} \in R^n$ is the normalized activation.

Batch Normalization



Ill-Conditioned Cost Function



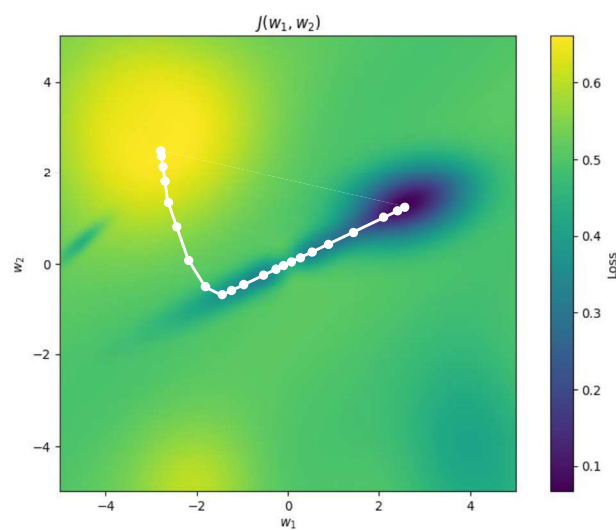
Ill-Conditioned Cost Function

- Problems:
 - Plateaus and saddle points $\Rightarrow$ Near-zero gradient
 - Ravines and sharp turns $\Rightarrow$ Oscillations or slow convergence
 - Local minima $\Rightarrow$ Stuck at high cost
- Solutions:
 - Mini-batches
 - Batch normalization
 - Adaptive optimization with momentum

Adaptive Optimization with Momentum

- **Momentum:** Accumulate speed in the gradient direction
 - $\Rightarrow$ Accelerates away from flat areas
 - $\Rightarrow$ Slows down when oscillating
 - $\Rightarrow$ Rolls past local minima
- **Adaptive learning rate scaling:** Adapted learning rate for every parameter
 - $\Rightarrow$ Moves slower in steep directions
 - $\Rightarrow$ Moves faster in flat directions
- Implemented in the **Adam** optimizer

Adam Optimizer



Overfitting

- Problem:
 - **Overfitting:** The model doesn't generalize to new, unseen data
- Solutions:
 - More data, including data augmentation
 - Less complex model, e.g. fewer neurons per layer
 - Regularization:
 - Early stopping
 - L1, L2 and weight decay
 - Dropout

Regularization: L1 and L2

- L2 regularization:

$$J'(\mathbf{W}, \mathbf{b}) = J(\mathbf{W}, \mathbf{b}) + \alpha \sum_{i,j} W_{i,j}^2$$

- L1 regularization:

$$J'(\mathbf{W}, \mathbf{b}) = J(\mathbf{W}, \mathbf{b}) + \alpha \sum_{i,j} |W_{i,j}|$$

regularization strength
(hyperparameter)

Weight Decay

- **Weight decay:**

$$\mathbf{W}' = (1 - \eta \lambda) \mathbf{W}$$

regularization strength
(hyperparameter)

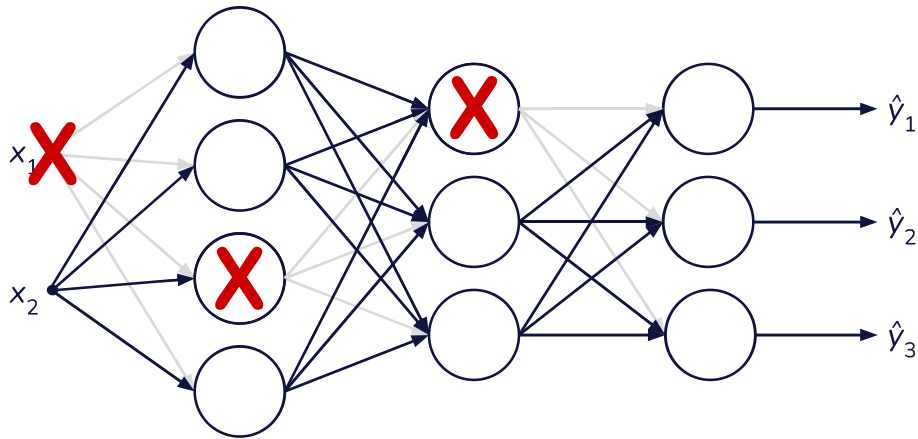


- Plain gradient descent $\Rightarrow$ Same effect as L2 regularization
- Adam:
 - $\Rightarrow$ L2 penalty term affects momentum and adaptation
 - $\Rightarrow$ Weight decay does not affect momentum and adaptation

Dropout

- **Dropout:** Disabling neurons randomly during training
 - Changes every training step
 - Only active during training, not inference
- **Dropout rate:** The probability of a neuron to be dropped
 - Requires tuning for every layer

Dropout



Transfer Learning

How to repurpose existing models

Transfer Learning

- Problem: Training deep models from scratch requires a lot of data
- **Transfer learning:** Reusing layers of an existing similar model
- Example: Personal face detector
 - Reuse a model that determines if there are people in an image

Transfer Learning

- A deep model detects low-level features in the first layers...
- ... and high-level features in the last layers

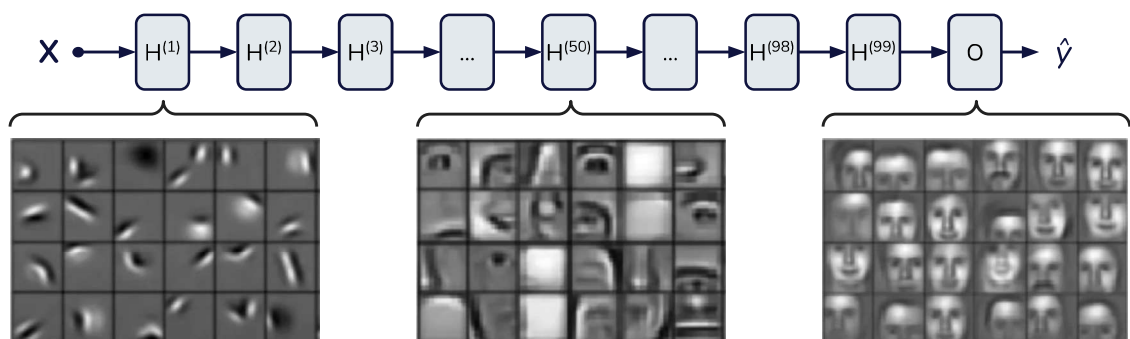


Image credit: <http://introtodeeplearning.com>

Transfer Learning

1. Start with a pretrained model
2. Remove the last few layers to obtain an advanced feature extractor
3. Attach new untrained layers
4. Train the new layers on own data, keeping the existing ones frozen

